

PROGRAMAÇÃO ORIENTADA A OBJETOS

ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Lázaro Pereira de Oliveira



Parte 04
Funções / Métodos

Prof. Lázaro Pereira de Oliveira

O QUE VEREMOS:

1) Métodos com Instância
2) Métodos com Herança

NO MESMO PACOTE

CALCULADORA COM USO DE DUAS CLASSES

NO MESMO PACOTE



Prof. Lázaro Oliveira

Exercício Calculadora – com uso de classe externa – no mesmo pacote

Utilizando **métodos/funções** em Java, faça:

Desenvolva um programa que realize operações matemáticas básicas: soma, subtração, multiplicação e divisão. O programa deve permitir, ao usuário, escolher a operação desejada (em um menu) e, em seguida, solicitar dois números para realizar a operação selecionada.



Requisitos:

A. O programa deve ter funções separadas para cada operação: soma, subtração, multiplicação e divisão.

B. O programa deve exibir um menu para o usuário escolher a operação desejada, a saber: 1-Somar; 2-Subtrair; 3-Multiplicar; 4-Dividirne e; 5-Sair.

C. Após a escolha da operação, o programa deve solicitar dois números e usar a função correspondente para calcular o resultado.

D. O programa deve exibir o resultado da operação.

E. O programa deve lidar com a divisão por zero, mostrando uma mensagem adequada se essa operação for escolhida.

F. O programa terá fim quando o usuário escolher a opção 5 (Sair)

G. Todas as funções (soma, subtração, multiplicação e divisão) devem estar escritas em uma classe denominada “Lib_Calculadora”, ao passo que o método principal (programa chamador) deverá estar escrito na classe “Calculadora”.

H. Considere as classes “Lib_Calculadora” e “Calculadora” **DENTRO DO MESMO PACOTE**.

Prof. Lázaro Oliveira

Exercício Calculadora – com uso de classe externa – no mesmo pacote

Quando se tem **as duas classes dentro do mesmo pacote**, **não é preciso importar nada manualmente**, pois **o Java já "enxerga" qualquer classe que esteja no mesmo pacote**.

Basta instanciar a classe **lib_calculadora** dentro da classe **calculadora**, ou então usar **herança** (se fizer sentido).

Não **existe import entre classes do mesmo pacote**. O import só é necessário quando você vai usar classes que estão em **pacotes diferentes**.

Exemplo com Instância:

```
package FuncoesMetodos;
public class calculadora {
    public static void main(String[] args) {
        lib_calculadora calc = new lib_calculadora();

        int soma = calc.somar(10, 5);
        int mult = calc.multiplicar(3, 4);

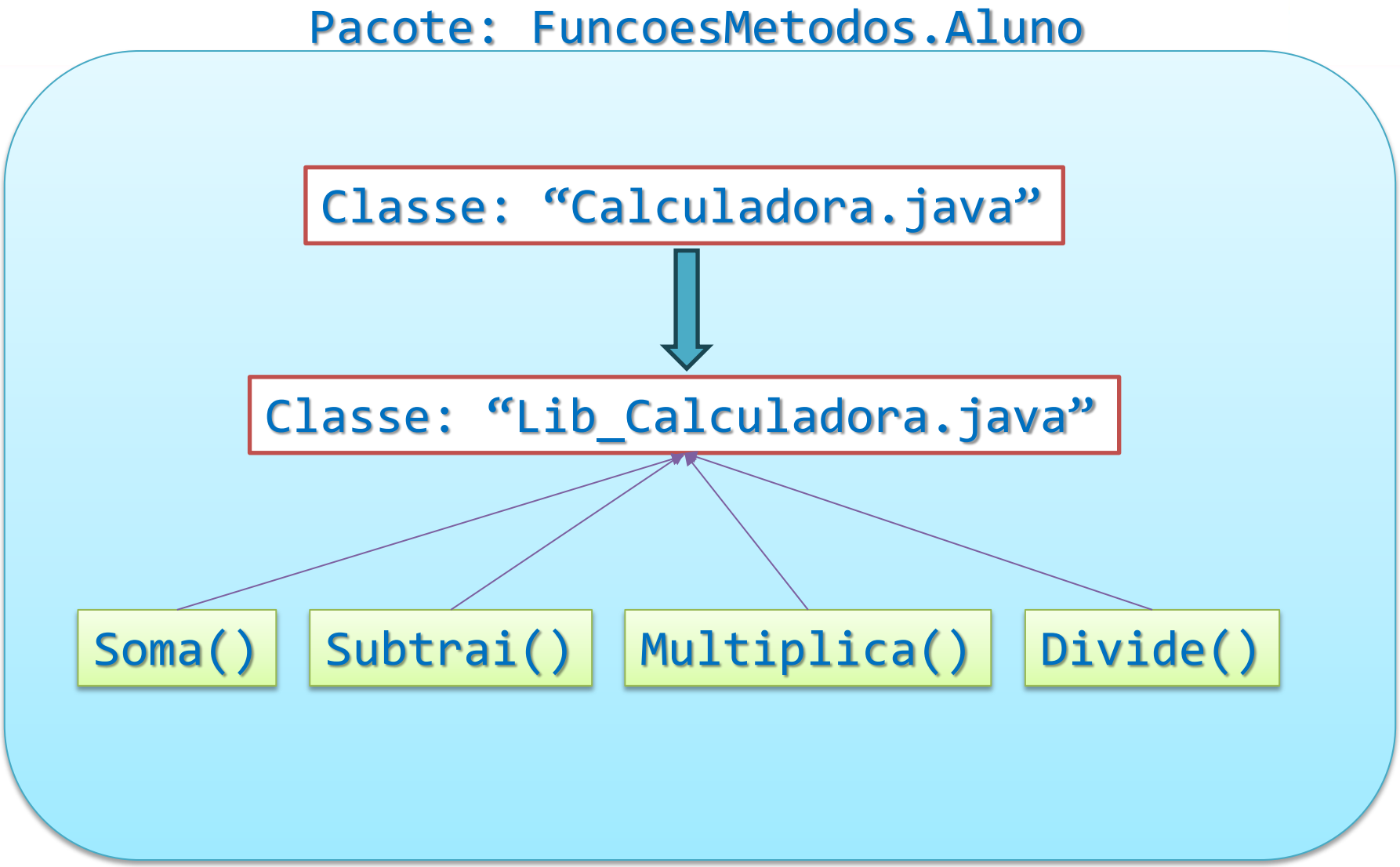
        System.out.println("Soma: " + soma);
        System.out.println("Multiplicação: " + mult);
    }
}
```

Exemplo com Herança:

```
package FuncoesMetodos;
public class calculadora extends lib_calculadora {
    public static void main(String[] args) {

        System.out.println("Soma: " + somar(10, 5));
        System.out.println("Multiplicação: " + multiplicar(3, 4));
    }
}
```

Exercício Calculadora – com uso de classe externa – no mesmo pacote



Essa Classe (Lib_Calculadora.java) será usada por todas as versões a seguir

```java
// #Lib_Calculadora.java
package _07_FuncoesMetodos.Calculadora;
public class Lib_Calculadora {
    public static double somar(double num1, double num2){
        return num1 + num2;
    }

    public static double subtrair(double num1, double num2){
        return num1 - num2;
    }

    public static double multiplicar(double num1, double num2){
        return num1 * num2;
    }

    public static double dividir(double num1, double num2){
        return num1 / num2;
    }
}
```

Prof. Lázaro Oliveira

Classe Calculadora com uso de duas classes:

Sem Instância e Sem Herança



Prof. Lázaro Oliveira

#Parte 1

```java
// #Calculadora_SemInstanciaSemHeranca.java
package _07_FuncoesMetodos.Calculadora;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Calculadora_SemInstanciaSemHeranca {
    public static int menu(){
        int op;
        Scanner entrada = new Scanner(System.in);
        try{
            System.out.println("---------------------------------");
            System.out.println("MENU PRINCIPAL:");
            System.out.println("1. Soma");
            System.out.println("2. Subtração");
            System.out.println("3. Multiplicação");
            System.out.println("4. Divisão");
            System.out.println("5. Sair\n");
            System.out.print  ("Digite o número da operação: ");
            op = entrada.nextInt();
            System.out.println("---------------------------------");
            //---------------------------------
            if(op < 1 || op > 5){
                System.out.println("Opção inválida! Tente novamente");
                op=menu();
            }
            //---------------------------------
            return op;
        }
        catch (InputMismatchException e){ //Excessão: digita caracter no lugar de número
            System.out.println("Entrada inválida! Digite apenas número entre 1 e 5.");
            op=menu();
            return op;
        }
        catch (Exception e){ // captura qualquer tipo de excessão
            //exibe a mensagem e, também, o typo do erro que está na variável "e"
            System.out.printf("Erro: Ocorreu um erro inesperado (%s)\n", e);
            op=menu();
            return op;
        }
    }
```

Prof. Lázaro Oliveira

#Parte 2

```
42      public static void main(String[] args){
43          Scanner entrada = new Scanner(System.in);
44          int opcao;
45          double n1, n2,resultado;
46          opcao = menu();  //chama o método menu para montar o menu de opções
47
48          while (opcao != 5){
49              System.out.print("Digite o primeiro número: ");
50              n1 = entrada.nextDouble();
51              System.out.print("Digite o segundo número: ");
52              n2 = entrada.nextDouble();
                switch (opcao) {
54                  case 1: //somar
55                      resultado= Lib_Calculadora.somar(n1,n2);
56                      System.out.println("Resultado da Soma: " +resultado);
57                      break;
58
59                  case 2: //subtrair
60                      resultado=Lib_Calculadora.subtrair(n1,n2);
61                      System.out.println("Resultado da Subtração: " +resultado);
62                      break;
63
64                  case 3:  // multiplicar
65                      resultado=Lib_Calculadora.multiplicar(n1,n2);
66                      System.out.println("Resultado da Multiplicação: " +resultado);
67                      break;
68
69                  case 4: //dividir
70                      if(n2==0){
71                          System.out.println("Não é possível dividir por zero.");
72                          break;
73                      }else{
74                          resultado=Lib_Calculadora.dividir(n1,n2);
75                          System.out.println("Resultado da Divisão: " +resultado);
76                          break;
77                      }
78              }
79              opcao=menu();
80          }
81          System.out.println("Programa finalizado!");
82      }
83  }
```

Prof. Lázaro Oliveira

Classe Calculadora com uso de duas classes:

Com Instância



Prof. Lázaro Oliveira

#Parte 1

```java
// #Calculadora_ComInstancia.java
package _07_FuncoesMetodos.Calculadora;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Calculadora_ComInstancia {
    public static int menu(){
        int op;
        Scanner entrada = new Scanner(System.in);
        try{
            System.out.println("---------------------------------");
            System.out.println("MENU PRINCIPAL:");
            System.out.println("1. Soma");
            System.out.println("2. Subtração");
            System.out.println("3. Multiplicação");
            System.out.println("4. Divisão");
            System.out.println("5. Sair\n");
            System.out.print  ("Digite o número da operação: ");
            op = entrada.nextInt();
            System.out.println("---------------------------------");
            //---------------------------------
            if(op < 1 || op > 5){
                System.out.println("Opção inválida! Tente novamente");
                op=menu();
            }
            //---------------------------------
            return op;
        }
        catch (InputMismatchException e){ //Excessão: digita caracter no lugar de número
            System.out.println("Entrada inválida! Digite apenas número entre 1 e 5.");
            op=menu();
            return op;
        }
        catch (Exception e){ // captura qualquer tipo de excessão
            //exibe a mensagem e, também, o typo do erro que está na variável "e"
            System.out.printf("Erro: Ocorreu um erro inesperado (%s)\n", e);
            op=menu();
            return op;
        }
    }
```

Prof. Lázaro Oliveira

#Parte 2

```
42      public static void main(String[] args){
43          Scanner entrada = new Scanner(System.in);
44          Lib_Calculadora calc = new Lib_Calculadora();//instância da biblioteca
45
46          int opcao;
47          double n1, n2,resultado;
48          opcao = menu();  //chama o método menu para montar o menu de opções
49
50          while (opcao != 5){
51              System.out.print("Digite o primeiro número: ");
52              n1 = entrada.nextDouble();
53              System.out.print("Digite o segundo número: ");
54              n2 = entrada.nextDouble();
                switch (opcao) {
56                  case 1: //somar
                        resultado= calc.somar(n1,n2);
58                      System.out.println("Resultado da Soma: " +resultado);
59                      break;
60
61                  case 2: //subtrair
                        resultado=calc.subtrair(n1,n2);
63                      System.out.println("Resultado da Subtração: " +resultado);
64                      break;
65
66                  case 3:  // multiplicar
                        resultado=calc.multiplicar(n1,n2);
68                      System.out.println("Resultado da Multiplicação: " +resultado);
69                      break;
70
71                  case 4: //dividir
72                      if(n2==0){
73                          System.out.println("Não é possível dividir por zero.");
74                          break;
75                      }else{
                            resultado=calc.dividir(n1,n2);
77                          System.out.println("Resultado da Divisão: " +resultado);
78                          break;
79                      }
80              }
81              opcao=menu();
82          }
83          System.out.println("Programa finalizado!");
84      }
85  }
```

Prof. Lázaro Oliveira

Classe Calculadora com uso de duas classes:

Com Herança



Prof. Lázaro Oliveira

#Parte 1

```java
// #Calculadora_ComInstancia.java
package _07_FuncoesMetodos.Calculadora;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Calculadora_ComHeranca extends Lib_Calculadora {
    public static int menu(){
        int op;
        Scanner entrada = new Scanner(System.in);
        try{
            System.out.println("---------------------------------");
            System.out.println("MENU PRINCIPAL:");
            System.out.println("1. Soma");
            System.out.println("2. Subtração");
            System.out.println("3. Multiplicação");
            System.out.println("4. Divisão");
            System.out.println("5. Sair\n");
            System.out.print  ("Digite o número da operação: ");
            op = entrada.nextInt();
            System.out.println("---------------------------------");
            //-----------------------------------
            if(op < 1 || op > 5){
                System.out.println("Opção inválida! Tente novamente");
                op=menu();
            }
            //-----------------------------------
            return op;
        }
        catch (InputMismatchException e){ //Excessão: digita caracter no lugar de número
            System.out.println("Entrada inválida! Digite apenas número entre 1 e 5.");
            op=menu();
            return op;
        }
        catch (Exception e){ // captura qualquer tipo de excessão
            //exibe a mensagem e, também, o typo do erro que está na variável "e"
            System.out.printf("Erro: Ocorreu um erro inesperado (%s)\n", e);
            op=menu();
            return op;
        }
    }
```

Prof. Lázaro Oliveira

#Parte 2

```
42      public static void main(String[] args){
43          Scanner entrada = new Scanner(System.in);
44          int opcao;
45          double n1, n2,resultado;
46          opcao = menu();  //chama o método menu para montar o menu de opções
47
48          while (opcao != 5){
49              System.out.print("Digite o primeiro número: ");
50              n1 = entrada.nextDouble();
51              System.out.print("Digite o segundo número: ");
52              n2 = entrada.nextDouble();
                switch (opcao) {
54                  case 1: //somar
55                      resultado= somar(n1,n2);
56                      System.out.println("Resultado da Soma: " +resultado);
57                      break;
58
59                  case 2: //subtrair
60                      resultado=subtrair(n1,n2);
61                      System.out.println("Resultado da Subtração: " +resultado);
62                      break;
63
64                  case 3:  // multiplicar
65                      resultado=multiplicar(n1,n2);
66                      System.out.println("Resultado da Multiplicação: " +resultado);
67                      break;
68
69                  case 4: //dividir
70                      if(n2==0){
71                          System.out.println("Não é possível dividir por zero.");
72                          break;
73                      }else{
74                          resultado=dividir(n1,n2);
75                          System.out.println("Resultado da Divisão: " +resultado);
76                          break;
77                      }
78              }
79              opcao=menu();
80          }
81          System.out.println("Programa finalizado!");
82      }
83  }
```

Prof. Lázaro Oliveira

Utilizando métodos/Funções, faça um programa para ler o nome e duas notas de uma turma com “n” alunos. Para cada aluno, informar sua média, situação e grau. Após exibir as informações do aluno, o programa deverá perguntar, ao usuário, se deseja continuar.

Situação:

“Aprovado”: para média maior ou igual a 6.
“Reprovado”: para média menor que 6.

Grau:

“Insuficiente”: Para média menor que 6.
“Regular”: para média entre 6 e 8 (exclusive)
“Bom”: para média entre 8 e 9 (exclusive)
“Excelente”: para média maior ou igual a 9.



Critério:

O programa deverá contar com quatro métodos:

1. exibirMedia()
2. calcularMedia()
3. exibirSituacao()
4. exibirGrau()

O programa deverá contar com duas classes diferentes, a saber:

1. **Classe Lib_Aluno (conterá todos os métodos)**
2. **Classe Aluno (conterá o método principal)**

Prof. Lázaro Oliveira

```java
// #Lib_Aluno.java

public class Lib_Aluno {
    public static double calcularMedia(double nota1, double nota2) {
        return (nota1 + nota2) / 2;
    }
    public static void exibirMedia(String nome, double media) {
        System.out.printf("A média do aluno %s é: %.2f \n", nome, media);
    }
    public static void exibirSituacao(double media){
        String situacao;
        if (media < 6){
            situacao = "Reprovado";
        }else{
            situacao = "Aprovado";
        }
        System.out.printf("Aluno %s \n",situacao);
    }
    public static void exibirGrau(double media){
        String grau;
        if (media < 6)
            grau = "Insuficiente";
        else if (media < 8)
            grau = "Regular";
        else if (media < 9)
            grau = "Bom";
        else
            grau = "Excelente";

        System.out.printf("Aluno obteve o grau %s \n",grau);
    }
}
```

Prof. Lázaro Oliveira

```java
// #Aluno.java
import java.util.InputMismatchException;
import java.util.Scanner;
// #Aluno.java
public class Aluno extends Lib_Aluno{
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        String nome, continuar="";
        double media, nota1, nota2;

        do {
            System.out.print("\nNome do aluno: ");
            nome = entrada.next(); //transforma a entrada em caixa baixa
            try{
                System.out.print("Primeira nota do aluno: ");
                nota1 = entrada.nextDouble();

                System.out.print("Segunda nota do aluno: ");
                nota2 = entrada.nextDouble();

                media = calcularMedia(nota1, nota2);
            }catch(InputMismatchException e){
                System.out.println("ERRO: Entrada inválida! Digite somente números para notas do aluno.");
                entrada.nextLine(); // Limpa o buffer do teclado
                continuar="s";
                continue;
            }
            exibirMedia(nome,media);
            exibirSituacao(media);
            exibirGrau(media);

            System.out.print("----------------------------- \n");

            System.out.print("Deseja continuar? (s/n): ");
            continuar = entrada.next();
            entrada.nextLine(); // Limpa o buffer do teclado
        } while (continuar.equalsIgnoreCase("s"));

        System.out.println("Programa encerrado.");
        entrada.close();
    }
}
```

Prof. Lázaro Oliveira

Prof. Lázaro Pereira de Oliveira

O QUE VEREMOS:

CLASSES

EM PACOTES DIFERENTES

CLASSES EM PACOTES DIFERENTES

Dando um Passo Adiante: Expandindo as Fronteiras do Nosso Código

Até agora, nossas classes conversavam muito de perto, como vizinhas que moram na mesma rua (dentro do **mesmo pacote**). Elas tinham acesso fácil umas às outras.

A partir de agora, nosso projeto vai crescer! Vamos aprender como uma classe pode acessar e interagir com outra classe que mora em um "bairro" totalmente diferente, ou seja, em um **outro pacote**.

Para fazer essa comunicação acontecer de forma organizada e segura, vamos aprender a usar dois conceitos fundamentais do Java:

import: A palavra-chave que usamos para "chamar" e dar o caminho de uma classe que está em outro pacote.

Modificadores de Acesso (public, protected, private): As regras de visibilidade que definem o que pode ou não ser visto e acessado de fora do pacote.

Vamos ver na prática como essa estrutura funciona!

Prof. Lázaro Pereira de Oliveira

MODIFICADORES DE ACESSO: O QUE SÃO?

Na programação orientada a objetos, os **modificadores de acesso** funcionam como "porteiros" do nosso código. Eles definem o nível de visibilidade e segurança, determinando quem de fora pode enxergar ou usar as classes, atributos e métodos.

No Java, os três principais são:

public (Público): O nível mais aberto. Qualquer classe, em **qualquer pacote** do projeto, pode ver e usar esse código.

- ✓ *No nosso exercício a seguir:* Usaremos public na Lib_Aluno para que o *Pacote1 consiga enxergar os métodos* dela.

private (Privado): O nível mais fechado e seguro. O código só pode ser acessado **dentro da própria classe** onde foi criado. Ninguém de fora (nem do mesmo pacote) consegue ver.

- ✓ *Para que serve:* Protege dados sensíveis contra alterações acidentais de outras partes do sistema.

protected (Protegido): O nível intermediário. O código fica visível para classes que estão no **mesmo pacote** e também para classes filhas (herança), mesmo que estejam em pacotes diferentes.

Por que isso importa agora?

- Como mudamos a estrutura do nosso projeto para múltiplos pacotes, se esquecermos de colocar o modificador **public** na classe Lib_Aluno ou em seus métodos, o Java aplicará o acesso padrão (*Package-Private*). Se isso acontecer, o Pacote1 ficará "cego" e **não conseguirá acessar a biblioteca**, gerando um erro de compilação!

Prof. Lázaro Pereira de Oliveira

ENUNCIADO...

Você foi contratado para desenvolver o protótipo de um sistema escolar em **Java**. Para manter o código organizado e seguir as boas práticas de programação orientada a objetos e modularização, o projeto deve ser dividido em pacotes e utilizar métodos estáticos importados.

Crie um programa que leia os dados de vários alunos, calcule suas médias e exiba o status acadêmico de cada um, tratando possíveis erros de digitação do usuário.

Estrutura de Arquivos e Pacotes

- O projeto deve possuir dois pacotes distintos: **Pacote1** e **Pacote2**.
- No **Pacote2**, crie uma biblioteca de funções chamada **Lib_Aluno.java**.
- No **Pacote1**, crie a classe principal chamada **Aluno.java**, que conterá o método main e fará a importação estática dos métodos da biblioteca.

Prof. Lázaro Pereira de Oliveira

Biblioteca de Métodos (Pacote2.Lib_Aluno.java) **...ENUNCIADO**

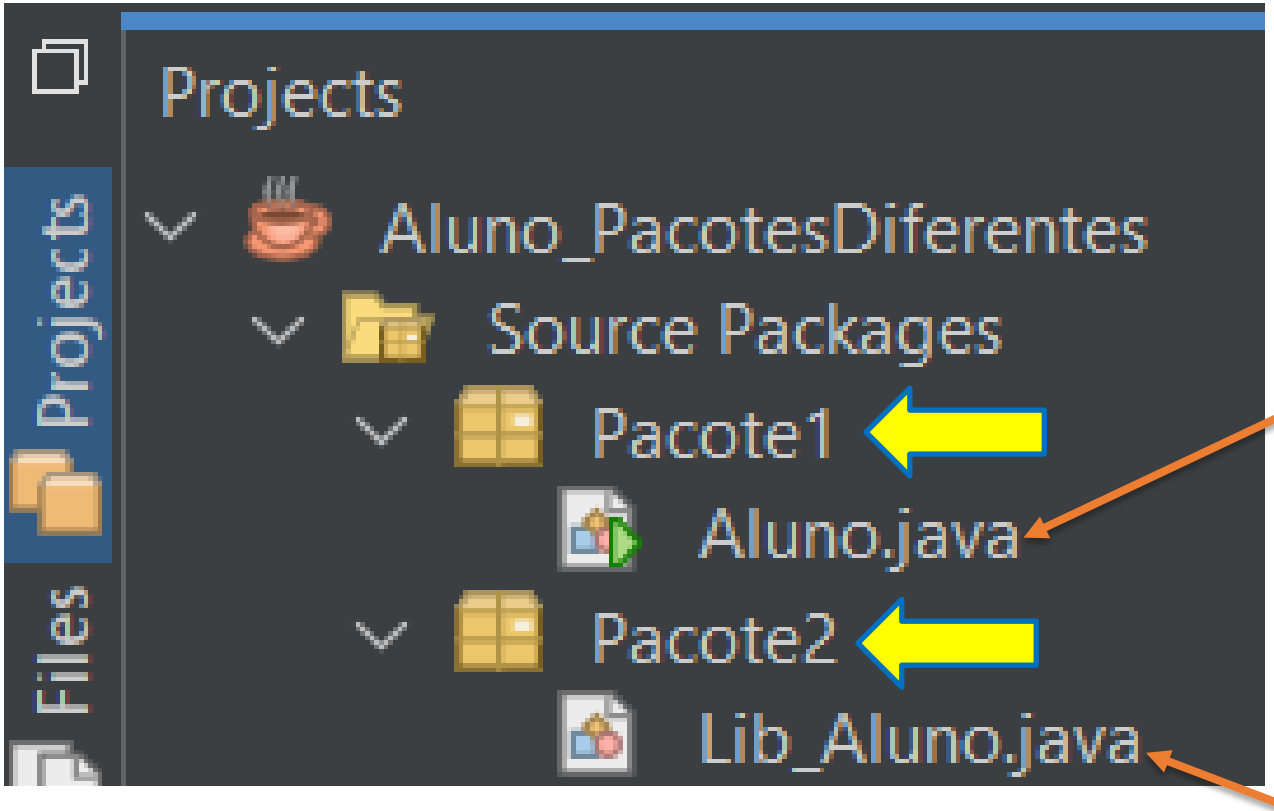
Implemente os seguintes métodos estáticos e públicos que serão utilizados pelo programa principal:

1) **calcularMedia(double nota1, double nota2)**: Recebe duas notas e retorna a média aritmética delas.
2) **exibirMedia(String nome, double media)**: Exibe uma mensagem na tela informando o nome do aluno e sua respectiva média (Ex: *"A lib_média do aluno [Nome] é: [Média]"*).
3) **exibirSituacao(double media)**: Avalia a média e exibe se o aluno está **"Aprovado"** (média maior ou igual a 6.0) ou **"Reprovado"** (média menor que 6.0).
4) **exibirGrau(double media)**: Converte a média numérica em um conceito de grau textual, seguindo os critérios:
   - ✓ Menor que 6.0 \(\rightarrow \) **"Insuficiente"**
   - ✓ De 6.0 até 7.9 \(\rightarrow \) **"Regular"**
   - ✓ De 8.0 até 8.9 \(\rightarrow \) **"Bom"**
   - ✓ Maior ou igual a 9.0 \(\rightarrow \) **"Excelente“**

Fluxo do Programa Principal (Pacote1.Aluno.java)

- O programa deve rodar dentro de um laço de repetição (do-while), permitindo que o usuário cadastre as notas de vários alunos consecutivamente. Ao final de cada aluno, pergunte se o usuário deseja continuar (s/n).
- Para cada aluno, o sistema deve solicitar o **Nome**, a **Primeira nota** e a **Segunda nota**.
- **Tratamento de Erros**: Utilize uma estrutura try-catch para capturar a exceção InputMismatchException. Caso o usuário digite um texto no lugar das notas, o sistema deve exibir uma mensagem de erro (*"ERRO: Entrada inválida! Digite somente números..."*), limpar o buffer do teclado e reiniciar o laço para o próximo aluno.

como o projeto ficará organizado nas pastas antes de codificar.



Classe principal com o método main()

Biblioteca com métodos estáticos

Prof. Lázaro Pereira de Oliveira

Lib_Aluno.java

```java
// #Lib_Aluno.java
package Pacote2;
public class Lib_Aluno {
    public static double calcularMedia(double nota1, double nota2) {
        return (nota1 + nota2) / 2;
    }
    public static void exibirMedia(String nome, double media) {
        System.out.printf("A lib_média do aluno %s é: %.2f \n", nome, media);
    }
    public static void exibirSituacao(double media){
        String situacao;
        if (media < 6){
            situacao = "Reprovado";
        }else{
            situacao = "Aprovado";
        }
        System.out.printf("Aluno %s \n",situacao);
    }
    public static void exibirGrau(double media){
        String grau;
        if (media < 6)
            grau = "Insuficiente";
        else if (media < 8)
            grau = "Regular";
        else if (media < 9)
            grau = "Bom";
        else
            grau = "Excelente";

        System.out.printf("Aluno obteve o grau %s \n",grau);
    }
}
```

Prof. Lázaro Pereira de Oliveira

Aluno.java

```java
// #Aluno.java
package Pacote1;
import java.util.InputMismatchException;
import java.util.Scanner;
import static Pacote2.Lib_Aluno.*;
/*
import static Pacote2.Lib_Aluno.calcularMedia;
import static Pacote2.Lib_Aluno.exibirGrau;
import static Pacote2.Lib_Aluno.exibirMedia;
import static Pacote2.Lib_Aluno.exibirSituacao;
import static Pacote2.Lib_Aluno.calcularMedia;
*/

public class Aluno{
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        String nome, continuar="";
        double media, nota1, nota2;

        do {
            System.out.print("\nNome do aluno: ");
            nome = entrada.next(); //transforma a entrada em caixa baixa
            try{
                System.out.print("Primeira nota do aluno: ");
                nota1 = entrada.nextDouble();

                System.out.print("Segunda nota do aluno: ");
                nota2 = entrada.nextDouble();

                media = calcularMedia(nota1, nota2);
            }catch(InputMismatchException e){
                System.out.println("ERRO: Entrada inválida! Digite somente números para notas do aluno.");
                entrada.nextLine(); // Limpa o buffer do teclado
                continuar="s";
                continue;
            }
                exibirMedia(nome,media);
                exibirSituacao(media);
                exibirGrau(media);

                System.out.print("---------------------------- \n");

                System.out.print("Deseja continuar? (s/n): ");
                continuar = entrada.next();
                entrada.nextLine(); // Limpa o buffer do teclado
            } while (continuar.equalsIgnoreCase("s"));

            System.out.println("Programa encerrado.");
            entrada.close();
        }
    }
```

Prof. Lázaro Pereira de Oliveira

```
Nome do aluno: Lázaro
Primeira nota do aluno: 10
Segunda nota do aluno: 5
A lib_média do aluno Lázaro é: 7,50
Aluno Aprovado
Aluno obteve o grau Regular
------------------------------
Deseja continuar? (s/n): n
Programa encerrado.
```

Método Estático versus não Estático

A principal diferença ao utilizar o modificador **static (estático)** em Java é que o método ou atributo passa a pertencer à classe em si, e não a um objeto (instância) específico criado a partir dela.

Para entender a diferença prática, vamos comparar o comportamento de métodos estáticos com métodos não-estáticos:

1. Chamada de Métodos (Como usamos no código anterior)

- ✓ **Com static (Método de Classe):** Você **não precisa criar um objeto** usando a palavra-chave new para chamar o método. Ele pode ser invocado diretamente pelo nome da classe ou, como feito no seu exercício, importado diretamente via import static.

```java
// Exemplo sem import estático: chamando direto pela classe
double media = Lib_Aluno.calcularMedia(8.0, 9.0);

// Exemplo com import estático (como está no seu código Aluno.java):
double media = calcularMedia(8.0, 9.0);
```

- ✓ **Sem static (Método de Instância):** Você é **obrigado a criar um objeto** na memória primeiro para depois conseguir usar o método.

```java
// Criando uma instância (objeto) da biblioteca
Lib_Aluno biblioteca = new Lib_Aluno();

// Chamando o método a partir do objeto criado
double media = biblioteca.calcularMedia(8.0, 9.0);
```

MODO ESTÁTICO VERSUS NÃO ESTÁTICO

Prof. Lázaro Pereira de Oliveira

2. Estado e Memória (Onde os dados ficam)

- ✓ **Métodos Estáticos:** São "funções puras". Eles recebem parâmetros, processam a informação e devolvem um resultado (ou exibem algo). Eles não guardam um histórico ou estado individual. A memória para eles é alocada uma única vez quando a classe é carregada.
- ✓ **Métodos Não-Estáticos:** Podem acessar e modificar as variáveis de instância (atributos do objeto). Por exemplo, se cada Aluno tivesse suas próprias notas salvas dentro do objeto (this.nota1), o método precisaria ser não-estático para ler essas variáveis específicas daquele aluno.

Característica	Com `static` (Estático)	Sem `static` (Instância)
Ligação	Vinculado à **Classe**.	Vinculado ao **Objeto**.
Criação (`new`)	**Não requer** instanciação.	**Obriga** a criação do objeto com `new` .
Acesso a Atributos	Só acessa outras variáveis/métodos estáticos.	Acessa variáveis globais do objeto (`this.nome`).
Uso Comum	Métodos utilitários, matemáticos ou bibliotecas de funções (ex: `Math.sqrt()`).	Modelagem de entidades do mundo real que possuem características próprias (ex: Clientes, Contas Bancárias).

Método Estático versus não Estático

POR QUE FOI USADO STATIC NO CÓDIGO ANTERIOR:

A classe Lib_Aluno funciona puramente como uma **biblioteca de funções utilitárias**.

O método calcularMedia(double n1, double n2), por exemplo, só precisa receber dois números, somar, dividir por dois e devolver o resultado.

Ele não precisa saber de qual aluno é essa nota e nem guardar essa informação na memória da classe.

Portanto, torná-lo static deixa o código mais limpo, rápido e consome menos memória, pois evita a criação desnecessária de objetos.

Prof. Lázaro Pereira de Oliveira

QUANDO DEVO USAR O MODO NÃO ESTÁTICO

Devemos usar o modo **não-estático** quando precisarmos representar objetos do mundo real que possuem características próprias (chamadas de atributos ou estado) e quando cada objeto precisar de dados individuais e independentes.

Enquanto o estático serve para funções gerais (como uma calculadora), o não-estático serve para criar "moldes" de coisas que têm identidade própria.

Exemplo Prático e Simples: Uma Conta Bancária

Imagine que temos uma classe ContaBancaria. Se ela fosse estática, existiria apenas uma conta para todo o sistema. Como não-estática, cada cliente pode ter a sua própria conta com saldos completamente diferentes.

1. A Classe (O Molde Não-Estático)

Repare que as variáveis e os métodos **não** possuem a palavra static

```java
public class ContaBancaria {
    // Atributos não-estáticos (cada conta terá o seu)
    String titular;
    double saldo;

    // Método não-estático (altera o saldo específico DESTA conta)
    public void depositar(double valor) {
        this.saldo += valor;
    }
}
```

Prof. Lázaro Pereira de Oliveira

Prof. Lázaro Pereira de Oliveira

2. O Programa Principal (Criando os Objetos)

Como a classe é não-estática, nós usamos o new para criar contas independentes na memória.

```
public class Sistema {
    public static void main(String[] args) {
        // Criando a conta do João
        ContaBancaria contaJoao = new ContaBancaria();
        contaJoao.titular = "João";
        contaJoao.depositar(500.0);

        // Criando a conta da Maria
        ContaBancaria contaMaria = new ContaBancaria();
        contaMaria.titular = "Maria";
        contaMaria.depositar(1200.0);

        // Cada objeto mantém seu próprio dado na memória!
        System.out.println("Saldo do João: R$ " + contaJoao.saldo);
        System.out.println("Saldo da Maria: R$ " + contaMaria.saldo);
    }
}
```

MODO ESTÁTICO VERSUS NÃO ESTÁTICO

Regra de ouro para decidir:

Pergunte a si mesmo: *"O comportamento desse método depende de quem está chamando ou ele funciona igual para qualquer um?"*

Se o saldo depende de **quem** é o dono da conta:

- ✓ **Não-estático** (Método de Objeto).

Se a fórmula matemática para calcular o imposto sobre o saldo é fixa e igual para todo mundo:

- ✓ **Estático** (Método de Classe).

Quando trabalhamos com o modo **não-estático**, estamos usando o Java exatamente para o que ele foi feito: criar classes que servem como "moldes" completos, agrupando **atributos** (características) e **métodos** (comportamentos) que operam juntos sobre esses dados.

Para resumir e fixar de forma definitiva:

Mundo Não-Estático (Orientado a Objetos): Você modela entidades. A classe tem variáveis (atributos) que definem o estado do objeto e métodos que usam essas variáveis. Cada cópia que você cria (new) é única.

Exemplo: Um objeto Carro que tem o atributo cor e o método pintar().

Mundo Estático (Procedural/Utilitário): Você cria apenas ferramentas soltas. A classe serve apenas como uma "caixa" para guardar funções que não pertencem a ninguém e não guardam dados.

Prof. Lázaro Pereira de Oliveira

PROGRAMAÇÃO ORIENTA A OBJETOS:

- ✓ Classes
- ✓ Atributos
- ✓ Métodos

Prof. Lázaro Pereira de Oliveira